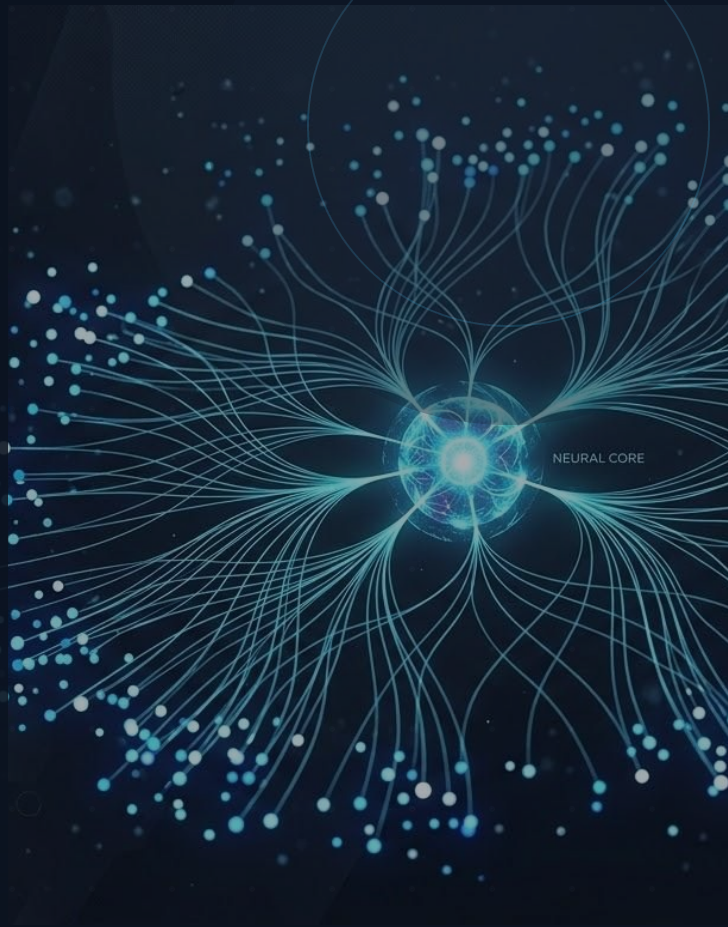
# Introdução a MCP e LLMs




Bruno Lima

# Capacidades Nativas vs. O Problema do Isolamento

## Capacidades Nativas das LLMs

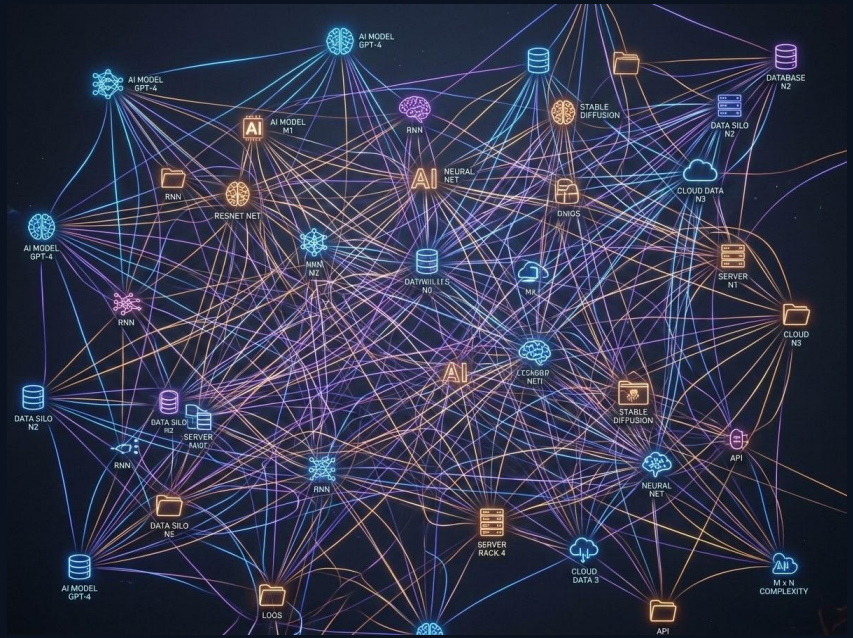
- Processamento e geração avançada de linguagem natural.
- Capacidade de raciocínio lógico, síntese de informações e tradução.
- Geração de código e sumarização de textos complexos.

## O Problema do Isolamento

- **Knowledge Cutoff:** Modelos possuem um limite temporal estrito de treinamento.
- **Falta de Contexto:** Incapacidade nativa de acessar dados privados ou informações em tempo real.
- **Inércia:** Dificuldade de executar ações no mundo real de forma padronizada.

# O Problema da Escalabilidade (M:N)

Atualmente, conectar múltiplos modelos de IA (M) a diversas fontes de dados (N) exige conectores específicos para cada combinação. Isso resulta em uma matriz complexa de M x N integrações personalizadas.




## Consequências do Modelo Atual

- **Custo Elevado:** Manutenção constante de múltiplos conectores e APIs.
- **Código Fragmentado:** Duplicação de esforços para diferentes provedores de LLMs.
- **Segurança:** Falta de padrões unificados para autenticação e controle de acesso.

# O Problema da Escalabilidade (M:N)




**Gargalo de Escalabilidade:** Sem um padrão unificado, adicionar 1 novo modelo ou 1 nova fonte exige refatorar e criar conectores para todas as combinações existentes.

# Model Context Protocol (MCP)

- **Protocolo Aberto:** O MCP é um protocolo aberto que padroniza como as aplicações fornecem contexto para as LLMs. Criado originalmente pela Anthropic, agora é disponibilizado como um padrão open-source para toda a comunidade.

- **O "USB-C" para IA:** O protocolo MCP unifica a conexão de dados e ferramentas às IAs.

A Solução

**Reduz a complexidade arquitetural de M x N para apenas M + N integrações.**

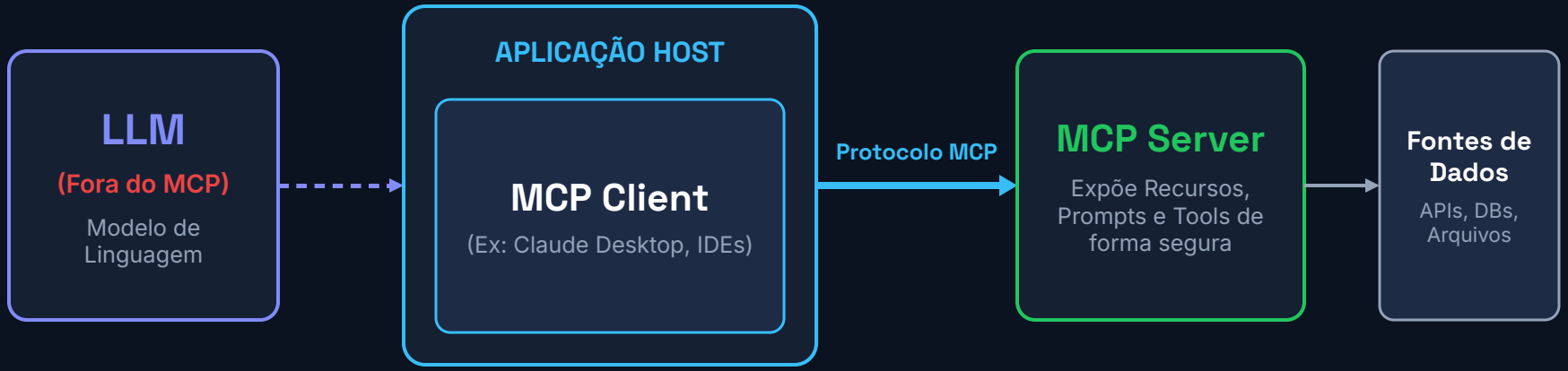
# Model Context Protocol (MCP)




**Arquitetura Escalável (M + N):** Com o MCP, adicionar 1 novo modelo ou fonte requer apenas 1 nova integração padronizada, sem refatorar o sistema existente.

# A Arquitetura MCP

O modelo de IA está estritamente isolado do Servidor




**Isolamento de Segurança:** O LLM opera de forma desacoplada, sem acesso direto aos dados. Todas as ações e contexto trafegam com controle rigoroso pela Aplicação Host e pelo Servidor MCP.

# Os Três Pilares Funcionais do MCP

## Prompts

Modelos de conversa e instruções pré-definidas expostas pelo servidor. Inclui slash commands e templates para análise de código, facilitando fluxos de trabalho.

## Resources

Dados lidos de forma segura e expostos ao LLM como contexto. Exemplos incluem logs de sistema, esquemas de banco de dados e arquivos JSON.

## Tools

Funções executáveis que a IA pode invocar para realizar ações no mundo real, como enviar uma mensagem no Slack ou criar uma issue no GitHub.

# Fluxo de Interação e Sequência no MCP

Como Host, Client e Server coordenam capacidades com o LLM




**O MCP padroniza a comunicação; a aplicação Host coordena a interação entre a LLM e as capacidades disponibilizadas pelos servidores.**

# Aplicações Práticas do MCP

- **Desenvolvimento de Software:** A IA atua como um assistente que analisa repositórios Git locais, consulta documentações técnicas e cria Pull Requests de forma autônoma.
- **Análise de Dados Empresariais:** Permite que usuários façam consultas diretas em linguagem natural a bancos de dados complexos, como Postgres ou Snowflake, sem escrever SQL.
- **Automação de Suporte e Operações:** Acelera operações lendo tickets diretamente no Zendesk ou Jira e executando scripts de diagnóstico automaticamente.

# O Ciclo de Execução de Ferramentas

01

## Descoberta

O cliente solicita a lista de ferramentas disponíveis ao servidor (via `tools/list`).

02

## Seleção

O LLM analisa o contexto do usuário e decide qual ferramenta usar, gerando os parâmetros necessários.

03

## Execução

O cliente invoca a ferramenta no servidor (via `tools/call`) com os parâmetros fornecidos pela IA.

04

## Resposta

O servidor retorna os resultados da execução, que o LLM sintetiza em uma resposta final para o usuário.

# Segurança e Governança no MCP

- **Controle de Acesso Local:** Os servidores MCP rodam diretamente no ambiente controlado pelo usuário ou pela empresa, mantendo os dados dentro da infraestrutura.
- **Human-in-the-Loop:** Ações sensíveis e destrutivas exigem confirmação explícita do usuário antes da execução, garantindo supervisão humana.
- **Isolamento de Credenciais:** O modelo de linguagem (LLM) recebe apenas os resultados das ações. Ele nunca tem acesso a chaves de API, tokens ou senhas de banco de dados.



# Demonstração: Claude Desktop + SQLite

## Cenário: Conectando um assistente de IA a um banco de dados local usando MCP.

claude_desktop_config.json — MCP Local Server Configuration

```
// Configuração do servidor SQLite no Claude Desktop
{ "mcpServers": { "sqlite": { "command": "uvx", "args": ["mcp-server-sqlite", "--db-path",
"./local_data.db"] } } }
> MCP Server Status: Ready | Active Protocol: Stdio
```

### Passo 1: Configuração

Edição do arquivo `claude_desktop_config.json` para registrar e mapear o servidor local.

### Passo 2: Inicialização

Inicialização automática do servidor MCP SQLite no ambiente local durante o boot do cliente.

### Passo 3: Interação em LN

Prompt em linguagem natural: **"Quais foram os 5 produtos mais vendidos este mês?"**

### Passo 4: Inspeção & Auditoria

Inspeção do log de execução para visualizar a query SQL gerada e os dados retornados com segurança.

# O Futuro do Ecossistema MCP

- **Adoção Rápida pela Indústria**

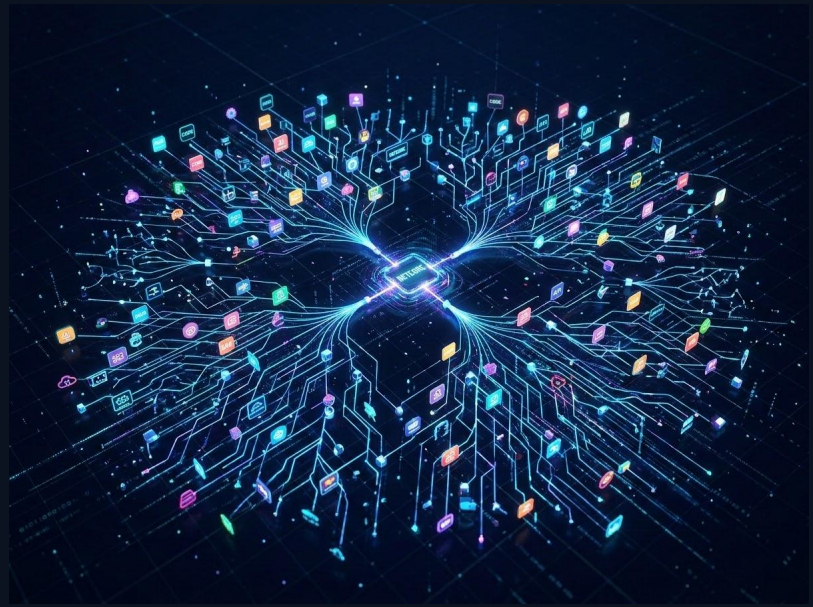
  Crescimento contínuo de servidores MCP prontos para uso em plataformas como GitHub, Slack, Google Drive e Postgres.

- **Evolução dos Agentes Autônomos**

  O MCP está se consolidando como a espinha dorsal de agentes inteligentes multi-ferramentas, permitindo fluxos de trabalho complexos.

- **Padronização Aberta**

  A neutralidade de fornecedor garante que desenvolvedores criem integrações uma única vez, funcionando em qualquer LLM que suporte o protocolo.



# Referências e Próximos Passos

Documentações oficiais e materiais técnicos para aprofundamento no MCP

## 1. Fundamentos do MCP

**Documentação & Especificação**

Arquitetura, comunicação entre Host, Client e Server.

`modelcontextprotocol.io`

## 2. Tools, Resources e Prompts

**Espec. Técnica de Capacidades**

Descoberta, solicitação e disponibilização de recursos por aplicações.

`github.com/modelcontextprotocol`

## 3. Implementação e SDKs

**Servidores de Referência e SDKs**

Exemplos práticos e SDKs oficiais em Python e TypeScript para criar servidores.

`python-sdk` | `typescript-sdk`

## 4. Segurança e Governança

**Uso Responsável & Diretrizes**

Considerações sobre permissões, confirmação humana e proteção de dados.

`modelcontextprotocol.io/docs/security`